Comparison of Linear Regression Models on The Canadian Housing Data

Linear Regression allows the prediction of continuous numerical target values based on categorical and numerical features.

In this project, I will:

- Perform data transformation and data cleaning.
- Explore different variations on linear regression on the Canadian Housing Dataset and compare their rmse outcomes.
- Use hyperparameter tuning and cross validation techniques.
- Offer possible explanations to the outcome.
- Provide next steps and improvements.

Load Libraries

In [95]:
```
import numpy as np
import pandas as pd

import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
import plotly.express as px


import plotly.graph_objects as go


from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline


from sklearn.linear_model import RidgeCV
from sklearn.linear_model import LassoCV
from sklearn.linear_model import ElasticNetCV
```

Functions

To plot the coefficient value of each feature onto a bar graph.

In [35]:
```
def plot_coef(X,model,name=None):

    plt.figure(figsize=(12,8))
    plt.bar(X.columns[2:],abs(model.coef_[2:]))
    plt.xticks(rotation=90)
    plt.ylabel("$coefficients$")
    plt.title(name)
    plt.show()
    print("R^2 on training  data ",model.score(X_train, y_train))
    print("R^2 on testing data ",model.score(X_test,y_test))
```

Calculate root mean squared error (RMSE)

In [36]:
```
def rmse(ytrue, ypredicted):
    return np.sqrt(mean_squared_error(ytrue, ypredicted))
```

In [37]:
```
def  plot_dis(y,yhat):

    plt.figure(figsize=(15,8))
    ax1 = sns.distplot(y, hist=False, color="r", label="Actual Value")
    sns.distplot(yhat, hist=False, color="b", label="Fitted Values" , ax=ax1)
    plt.legend()

    plt.title('Actual vs Fitted Values')
    plt.xlabel('Price (in dollars)')
    plt.ylabel('Proportion of Houses')

    plt.show()
    plt.close()
```

Loading Data

In [38]:
```
housing_df = pd.read_csv("cleaned_canada.csv")
housing_df.head()
```

Out[38]:

	City	Province	Latitude	Longitude	Price	Bedrooms	Bathrooms	Acreage
0	Revelstoke	BC	50.976585	-118.173149	839000.0	3.0	2.0	0.00
1	Boswell	BC	49.471870	-116.770195	1150000.0	3.0	2.0	0.32
2	West Kelowna	BC	49.825230	-119.603253	149000.0	2.0	1.0	0.00
3	Kelowna	BC	49.821860	-119.480143	1298000.0	5.0	4.0	0.69
4	Maple Ridge	BC	49.221673	-122.596637	759900.0	3.0	2.0	0.00

5 rows × 23 columns

Many categorical and numerical features can already be seen. The Price column will be our target for prediction.

In [39]:
```
housing_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 44896 entries, 0 to 44895
Data columns (total 23 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   City            44896 non-null  object 
 1   Province        44896 non-null  object 
 2   Latitude        44896 non-null  float64
 3   Longitude       44896 non-null  float64
 4   Price           44896 non-null  float64
 5   Bedrooms        44896 non-null  float64
 6   Bathrooms       44896 non-null  float64
 7   Acreage         44896 non-null  float64
 8   Property Type   44896 non-null  object 
 9   Square Footage  44896 non-null  float64
 10  Garage          44896 non-null  object 
 11  Parking         44896 non-null  object 
 12  Basement        14962 non-null  object 
 13  Exterior        17439 non-null  object 
 14  Fireplace       44896 non-null  object 
 15  Heating         38242 non-null  object 
 16  Flooring        15216 non-null  object 
 17  Roof            10052 non-null  object 
 18  Waterfront      44896 non-null  object 
 19  Sewer           44896 non-null  object 
 20  Pool            44896 non-null  object 
 21  Garden          44896 non-null  object 
 22  Balcony         44896 non-null  object 
dtypes: float64(7), object(16)
memory usage: 7.9+ MB
```

We can see from the above output that some features contain a lot of missing values. These must be either handled with a replacement or compelete removal.

In [40]:
```
housing_df.describe()
```

Out[40]:

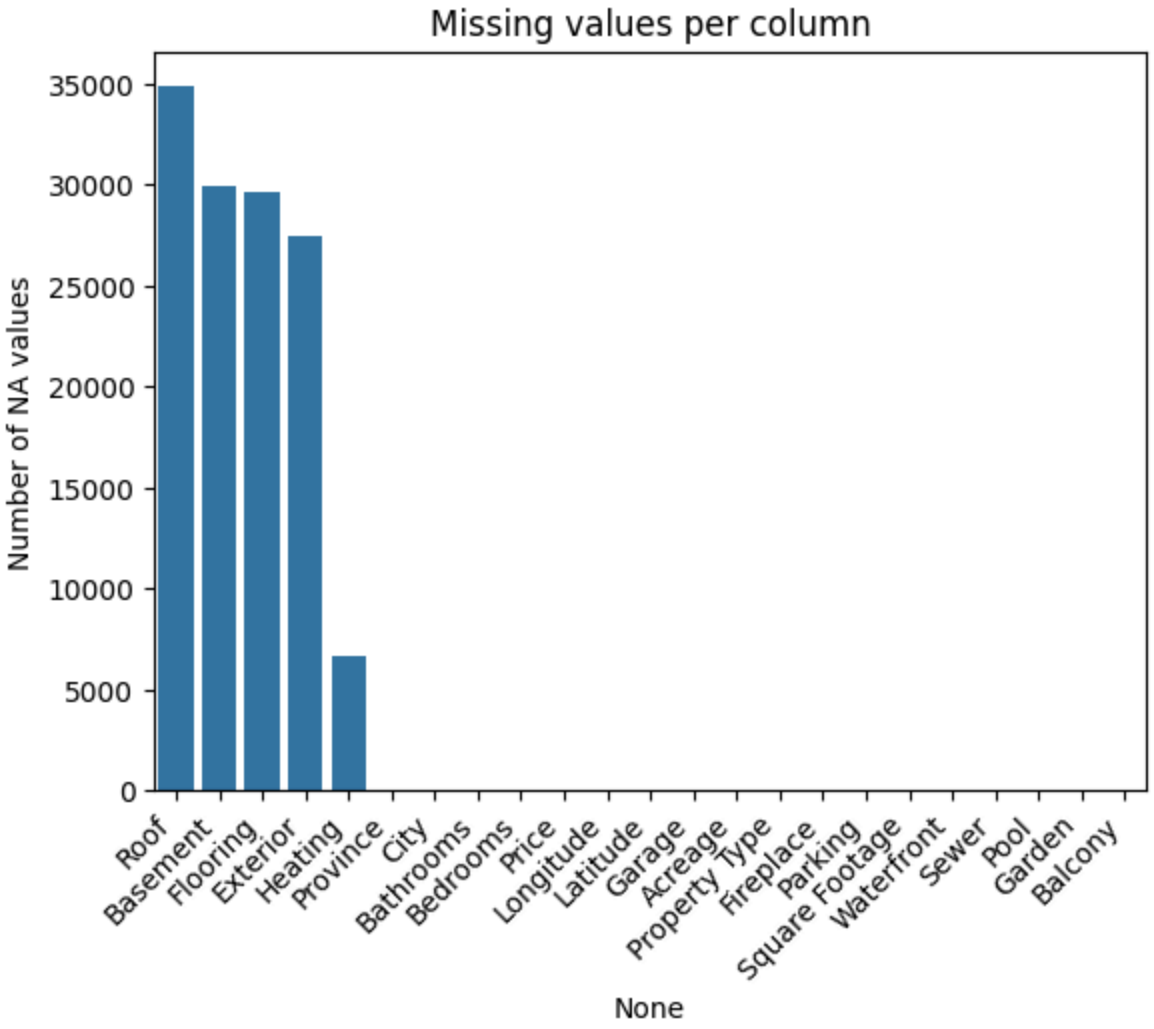
	Latitude	Longitude	Price	Bedrooms	Bathrooms	Acreage
count	44896.000000	44896.000000	4.489600e+04	44896.000000	44896.000000	44896.000000
mean	49.010267	-106.632760	1.070457e+06	3.214919	2.522652	2.485950
std	2.659066	22.270395	1.442961e+06	1.636517	1.423765	86.626599
min	42.045940	-135.856018	5.000000e+04	0.000000	0.000000	0.000000
25%	48.477511	-122.887882	3.999900e+05	2.000000	2.000000	0.000000
50%	49.217891	-119.395219	6.880000e+05	3.000000	2.000000	0.060000
75%	49.927914	-102.485638	1.200000e+06	4.000000	3.000000	0.180000
max	65.281488	-52.668600	5.880000e+07	37.000000	26.000000	8600.000000

We do not care about Latitude and Longitude in the above outcome since they are only positional features

Handling Missing Values

Let's visualize each column and the number of missing values they contain

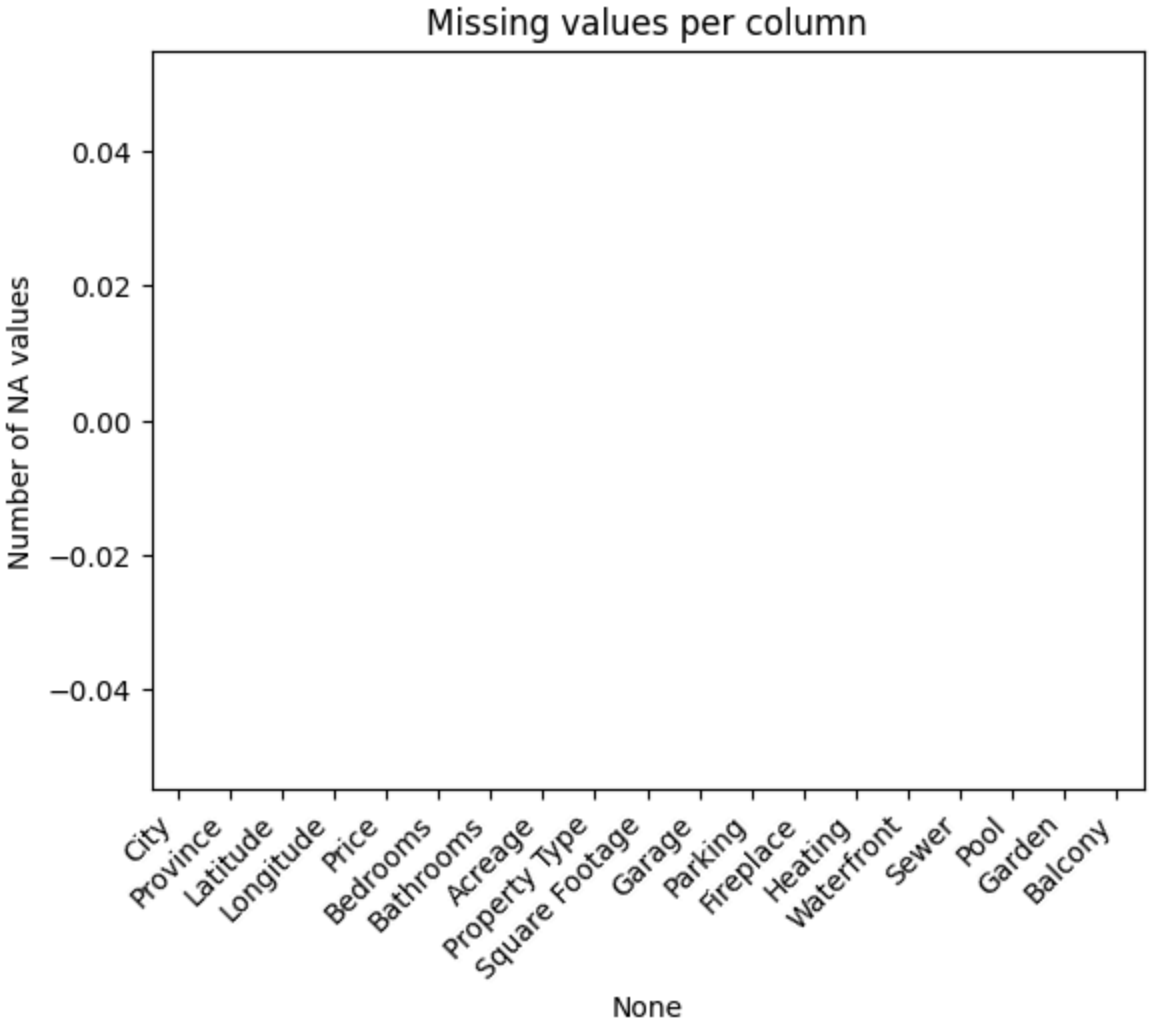
In [41]:
```
(housing_df.isna().sum().sort_values(ascending=False).pipe(lambda s: sns.barplot(x=
plt.xticks(rotation=45, ha='right')
plt.ylabel("Number of NA values")
plt.title("Missing values per column")
plt.show()
```



There are a lot of missing values in Roof, Basement, Flooring, Exterior, and Heating columns. For the heating column, since it is missing only 5000 columns and it could be a useful data point, we will impoute "Unkown" for missing values. As for the rest, they are missing too many values to keep. We will remove them.

In [42]:
```
housing_df = housing_df.drop(["Roof","Basement","Flooring","Exterior"], axis = 1)
housing_df['Heating'] = housing_df['Heating'].fillna("Unknown")
```

In [43]:
```
(housing_df.isna().sum().sort_values(ascending=False).pipe(lambda s: sns.barplot(x=
plt.xticks(rotation=45, ha='right')
plt.ylabel("Number of NA values")
plt.title("Missing values per column")
plt.show()
```



Looks like there are no more missing values in our dataset. We will print out the info below to double check with numbers.

In [44]:
```
housing_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 44896 entries, 0 to 44895
Data columns (total 19 columns):
 #   Column          Non-Null Count  Dtype  
---  ------          --------------  -----  
 0   City            44896 non-null  object 
 1   Province        44896 non-null  object 
 2   Latitude        44896 non-null  float64
 3   Longitude       44896 non-null  float64
 4   Price           44896 non-null  float64
 5   Bedrooms        44896 non-null  float64
 6   Bathrooms       44896 non-null  float64
 7   Acreage         44896 non-null  float64
 8   Property Type   44896 non-null  object 
 9   Square Footage  44896 non-null  float64
 10  Garage          44896 non-null  object 
 11  Parking         44896 non-null  object 
 12  Fireplace       44896 non-null  object 
 13  Heating         44896 non-null  object 
 14  Waterfront      44896 non-null  object 
 15  Sewer           44896 non-null  object 
 16  Pool            44896 non-null  object 
 17  Garden          44896 non-null  object 
 18  Balcony         44896 non-null  object 
dtypes: float64(7), object(12)
memory usage: 6.5+ MB
```

We no longer have missing values in our columns.

We will save categorical and numerical values in variables so we can access them easily later on.

In [45]:
```
mask = housing_df.dtypes == object
categorical_cols = housing_df.columns[mask]
categorical_cols
```

Out[45]:
```
Index(['City', 'Province', 'Property Type', 'Garage', 'Parking', 'Fireplace',
       'Heating', 'Waterfront', 'Sewer', 'Pool', 'Garden', 'Balcony'],
      dtype='object')
```

In [46]:
```
numeric_columns=list(set(housing_df.columns)-set(categorical_cols))
numeric_columns
```

Out[46]:
```
['Bedrooms',
 'Price',
 'Longitude',
 'Acreage',
 'Bathrooms',
 'Latitude',
 'Square Footage']
```

Overview Plots And EDA

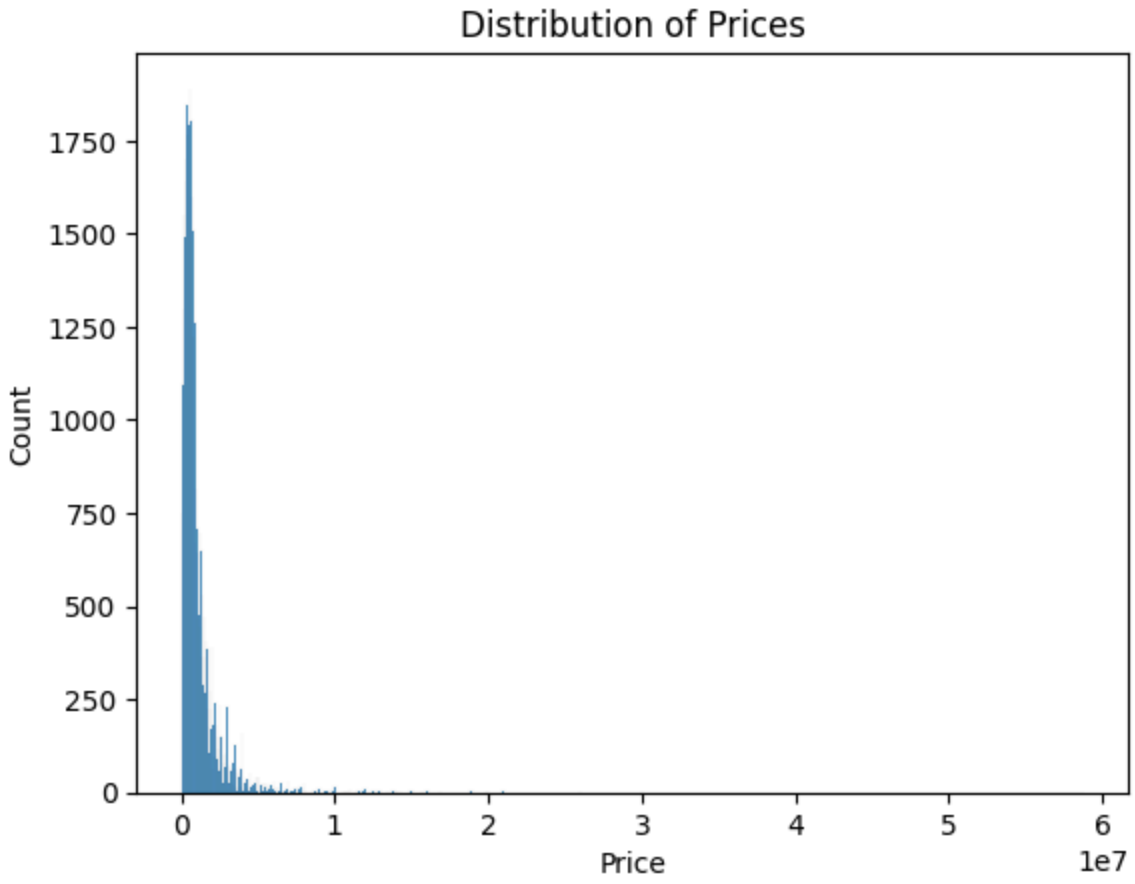
In [104…
```
fig = px.scatter_map(
    housing_df,
    lat="Latitude", lon="Longitude",
    color="Price",
    color_continuous_scale="Bluered_r",
    range_color=[min(housing_df['Price']), max(housing_df['Price'])],   # focus sca
    size_max=6, zoom=3,
    hover_data=["Price", "Square Footage"],    # add city + price to tooltip
    center={"lat":56,"lon":-106},  # roughly Canada center
    height=650
)

fig.update_layout(
    mapbox_style="carto-positron",
    margin=dict(l=0, r=0, t=40, b=0),
    title="Property Prices in Canada"
)
```

Interactive map: Zooming in will reveal the more expensive houses. The majority of the data is below 10M.

In [105…
```
fig = px.scatter_map(
    housing_df,
    lat="Latitude", lon="Longitude",
    color="Square Footage",
    color_continuous_scale="Bluered_r",
    range_color=[min(housing_df['Square Footage']), max(housing_df['Square Footage'
    size_max=6, zoom=3,
    hover_data=["Price", "Square Footage"],    # add city + price to tooltip
    center={"lat":56,"lon":-106},  # roughly Canada center
    height=650
)

fig.update_layout(
    mapbox_style="carto-positron",
    margin=dict(l=0, r=0, t=40, b=0),
    title="Square Footage of Properties in Canada"
)
```

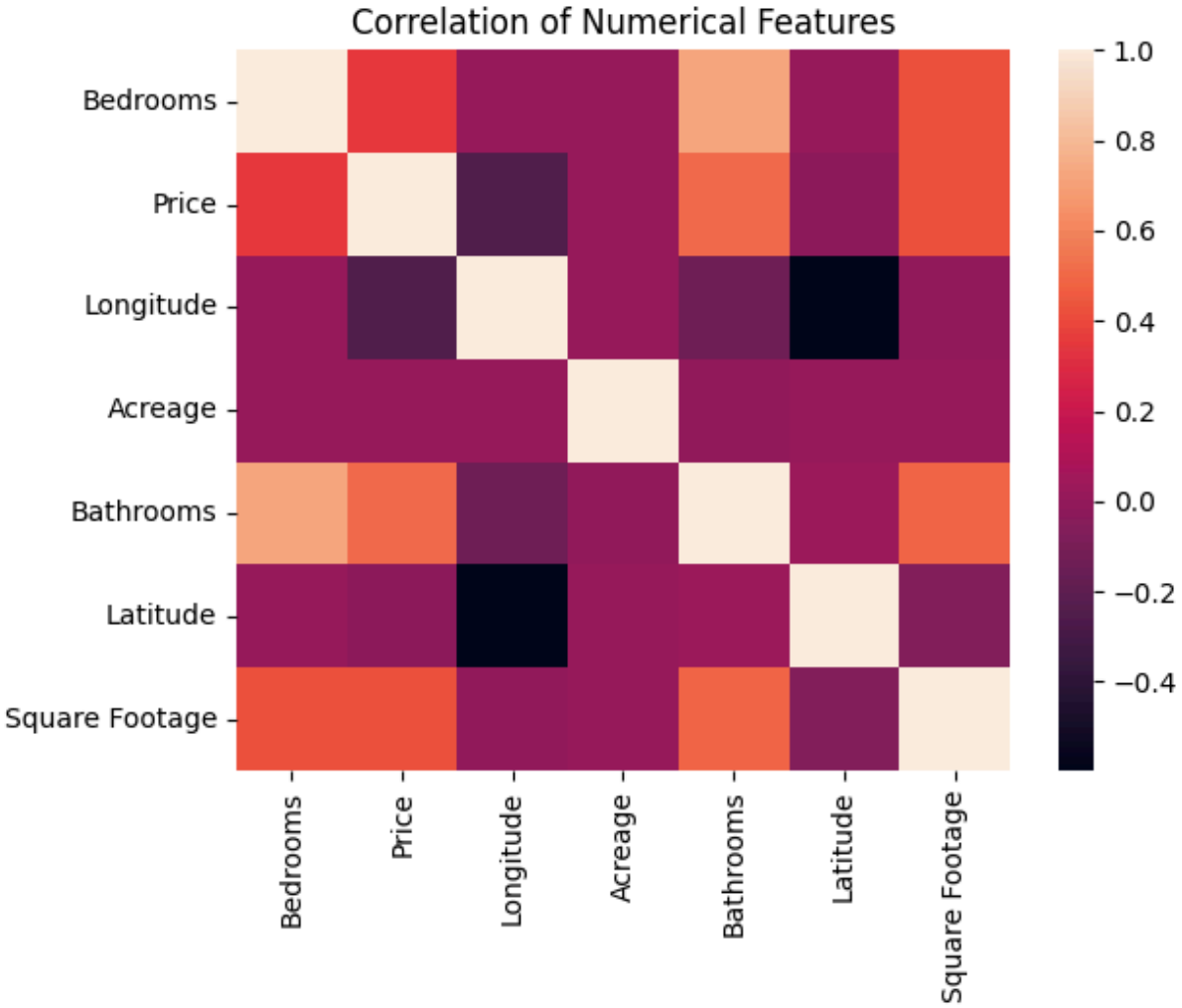
In [49]:
```
# plt.hist(data = housing_df, x="Price", bins = 100)
sns.histplot(data = housing_df, x = "Price")
plt.title("Distribution of Prices")
plt.xlabel("Price")
plt.ylabel("Count")
```

Out[49]:
```
Text(0, 0.5, 'Count')
```



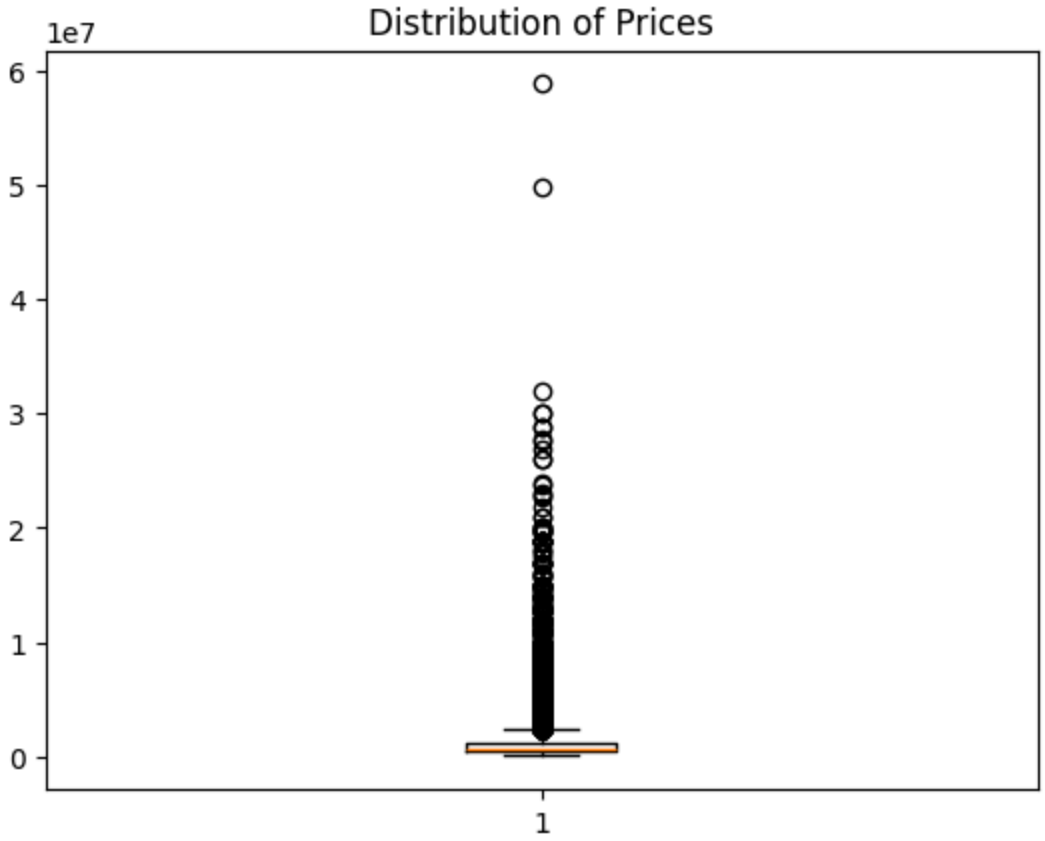
In [50]:
```
sns.heatmap(housing_df[numeric_columns].corr())
plt.title("Correlation of Numerical Features")
```

Out[50]:
```
Text(0.5, 1.0, 'Correlation of Numerical Features')
```



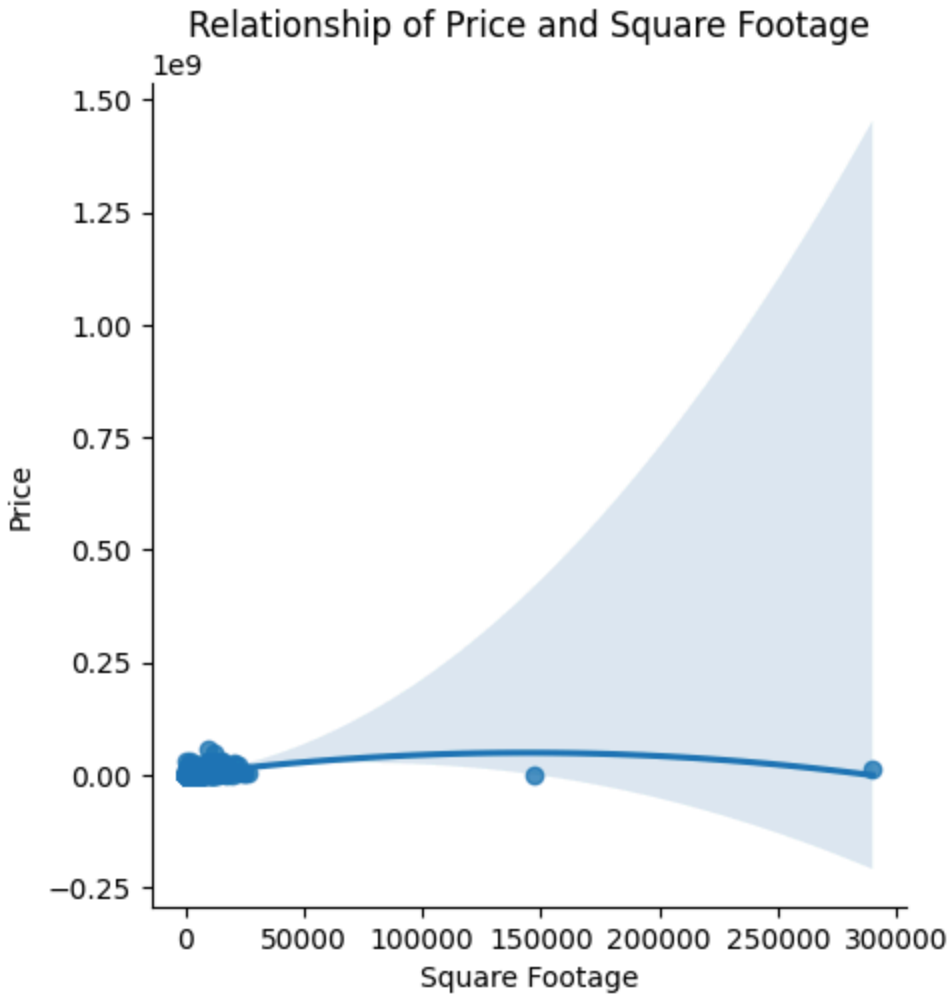
In [51]:
```
plt.boxplot(housing_df['Price'])
plt.title("Distribution of Prices")
```

Out[51]:
```
Text(0.5, 1.0, 'Distribution of Prices')
```



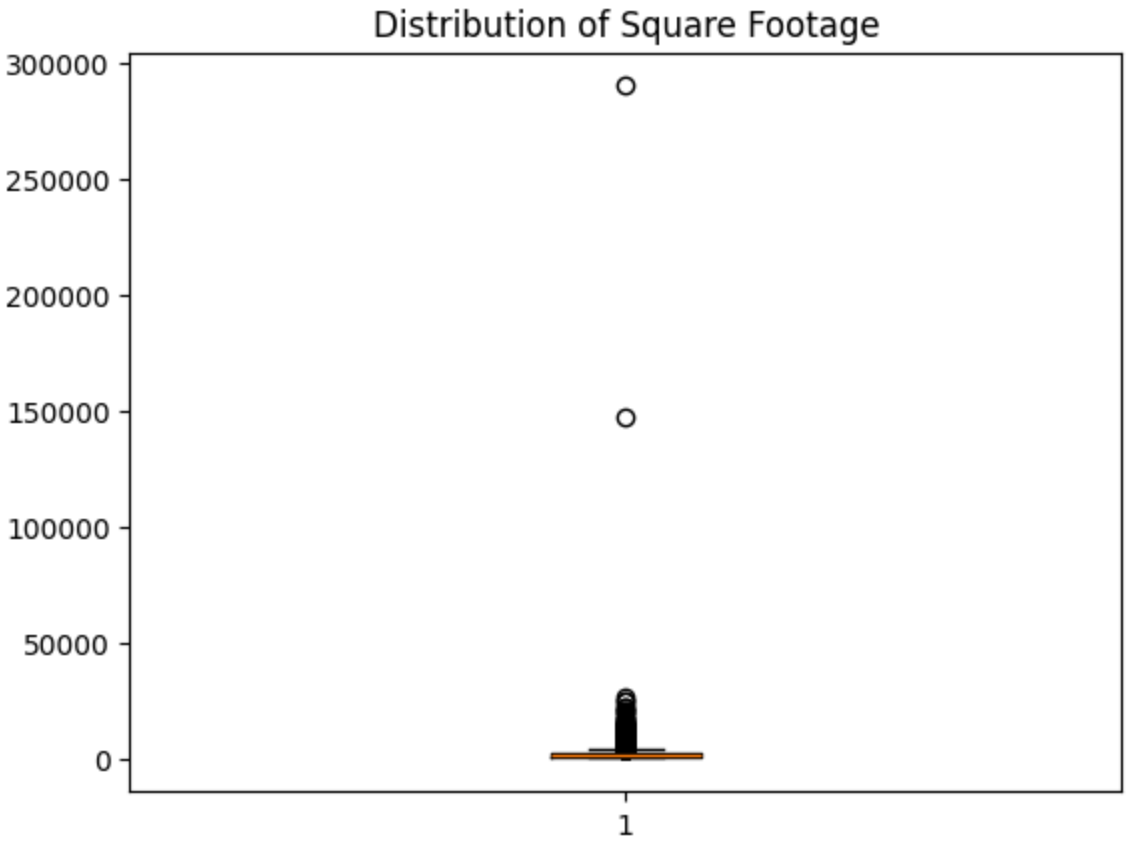
In [52]:
```
sns.lmplot(x = 'Square Footage', y = 'Price', data = housing_df, order=2)
plt.title("Relationship of Price and Square Footage")
```

Out[52]:
```
Text(0.5, 1.0, 'Relationship of Price and Square Footage')
```



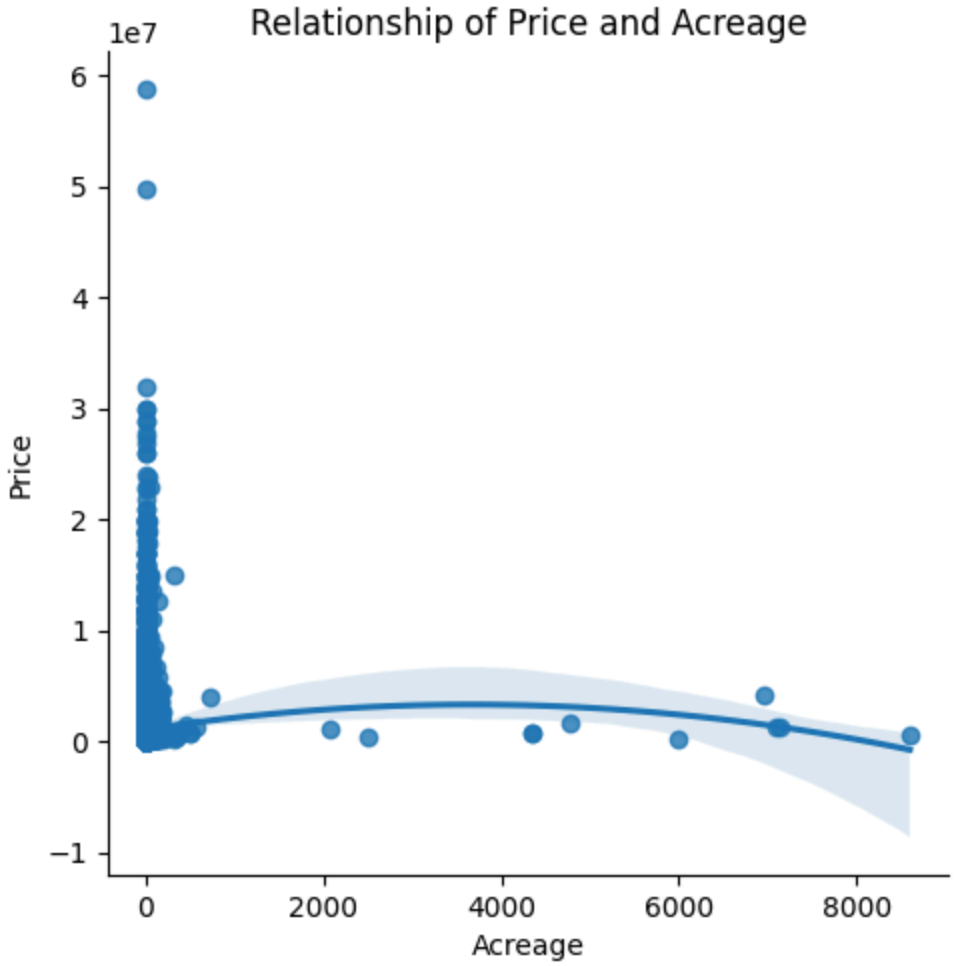
In [53]:
```
plt.boxplot(housing_df['Square Footage'])
plt.title("Distribution of Square Footage")
```

Out[53]:
```
Text(0.5, 1.0, 'Distribution of Square Footage')
```



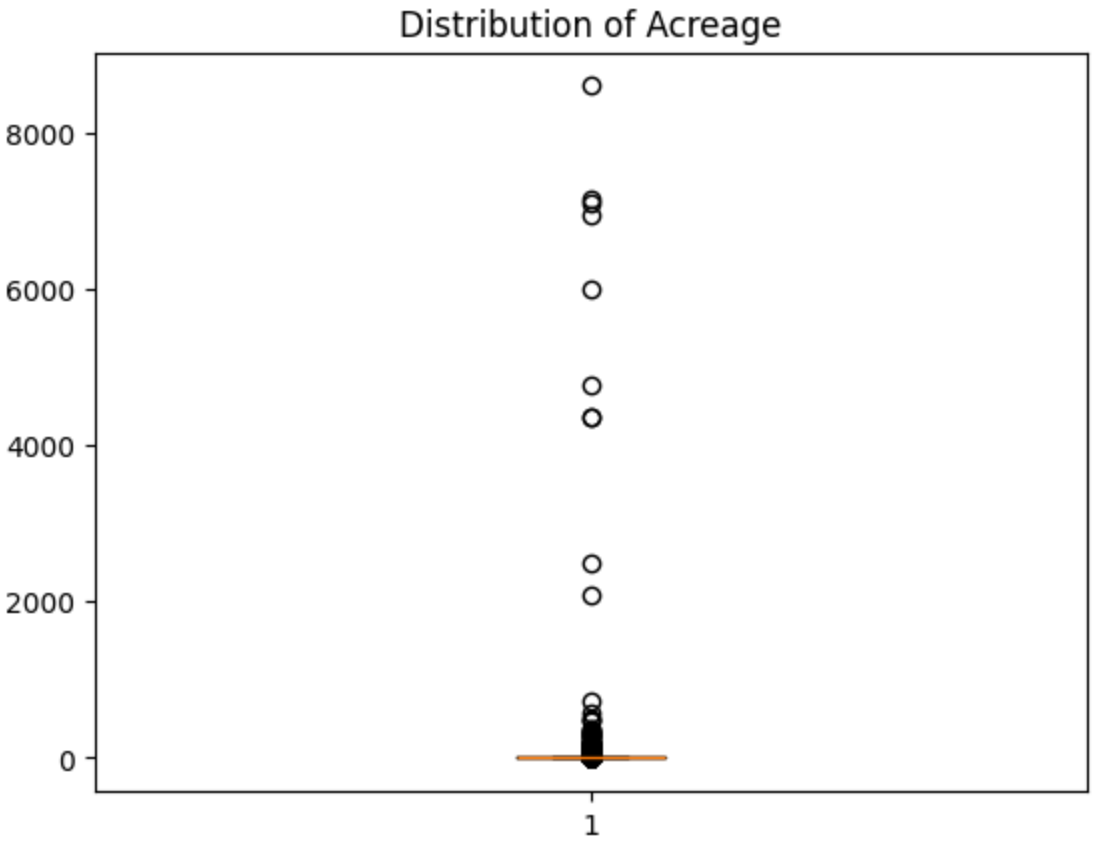
In [54]:
```
sns.lmplot(x = 'Acreage', y = 'Price', data = housing_df, order=2)
plt.title("Relationship of Price and Acreage")
```

Out[54]:
```
Text(0.5, 1.0, 'Relationship of Price and Acreage')
```



In [55]:
```
plt.boxplot(housing_df['Acreage'])
plt.title("Distribution of Acreage")
```

Out[55]:
```
Text(0.5, 1.0, 'Distribution of Acreage')
```



Calculating IQR to remove outliers in square footage and acreage features.

In [56]:
```
Q1 = housing_df['Square Footage'].quantile(0.25)
Q3 = housing_df['Square Footage'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print(f"Original shape: {housing_df.shape}")

housing_df = housing_df[(housing_df['Square Footage'] >= lower_bound) & (housing_df

print(f"New shape: {housing_df.shape}")
```

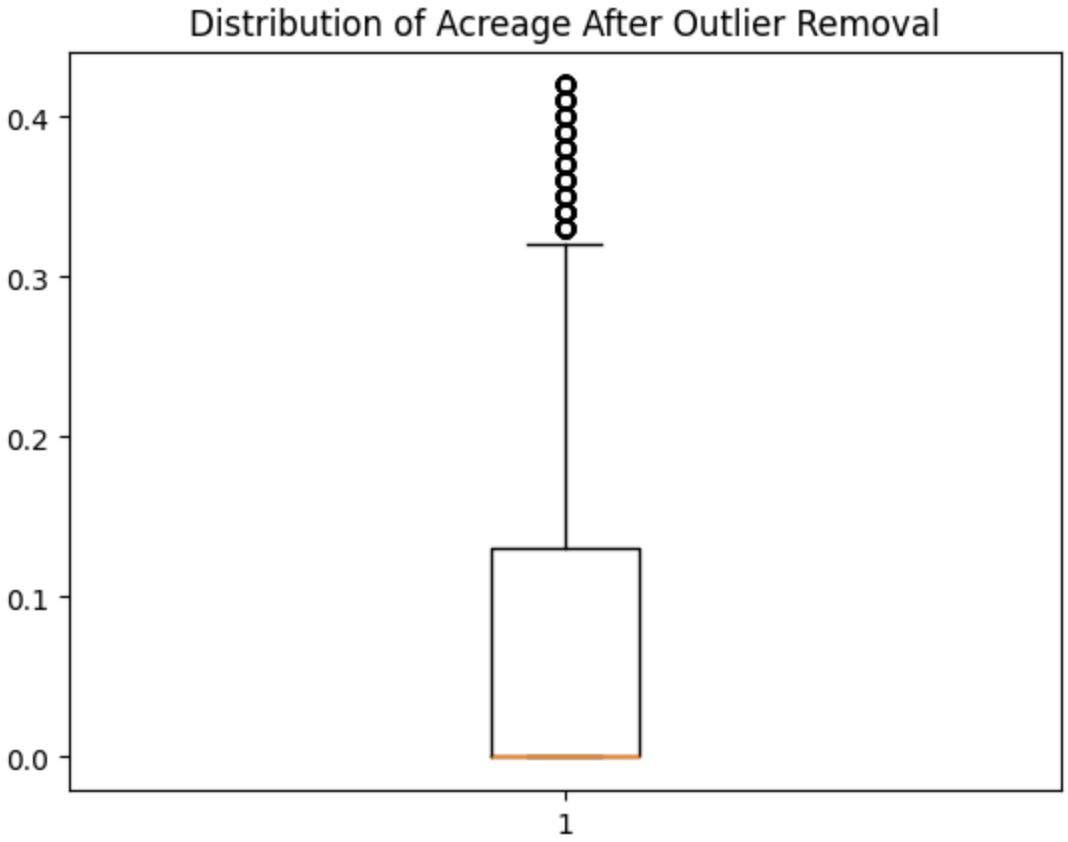
```
Original shape: (44896, 19)
New shape: (42529, 19)
```

In [57]:
```
Q1 = housing_df['Acreage'].quantile(0.25)
Q3 = housing_df['Acreage'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
print(f"Original shape: {housing_df.shape}")

housing_df_AcreageOutlierRemoval = housing_df[(housing_df['Acreage'] >= lower_bound

print(f"New shape: {housing_df_AcreageOutlierRemoval.shape}")
```

```
Original shape: (42529, 19)
New shape: (37596, 19)
```

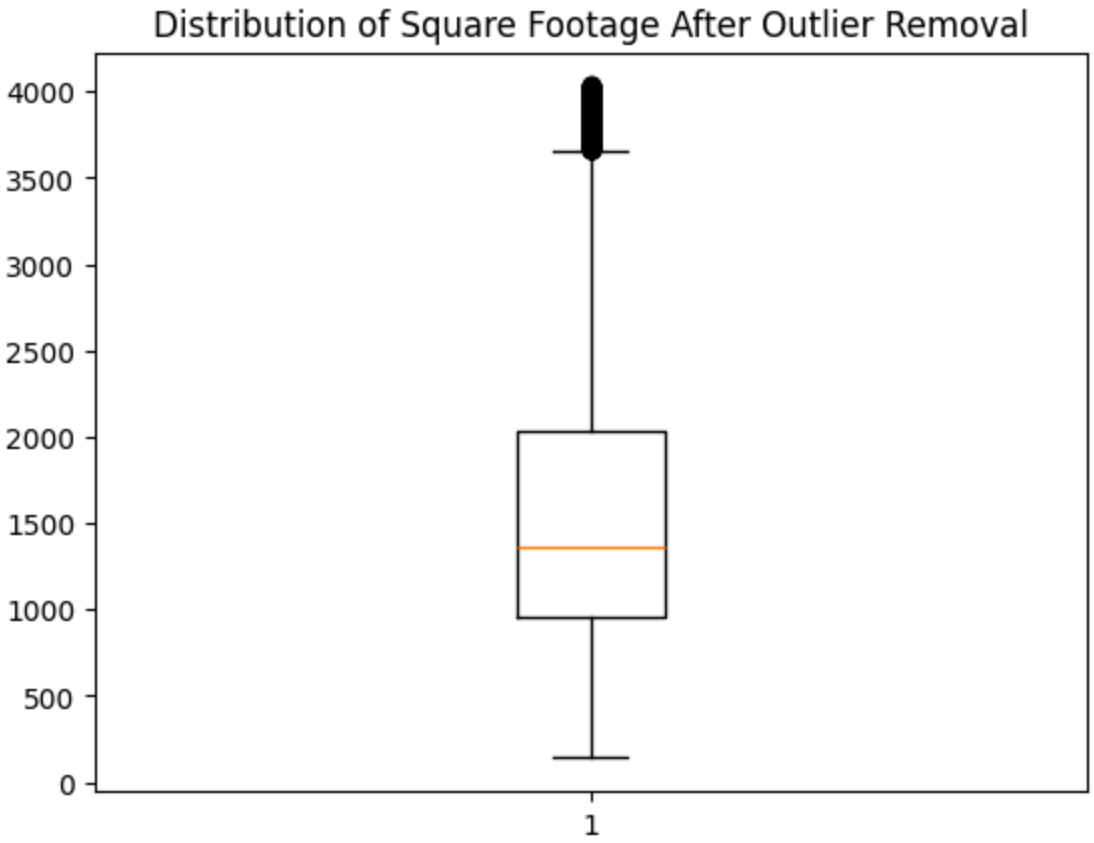
In [58]:
```
plt.boxplot(housing_df_AcreageOutlierRemoval['Acreage'])
plt.title("Distribution of Acreage After Outlier Removal")
```

Out[58]:
```
Text(0.5, 1.0, 'Distribution of Acreage After Outlier Removal')
```



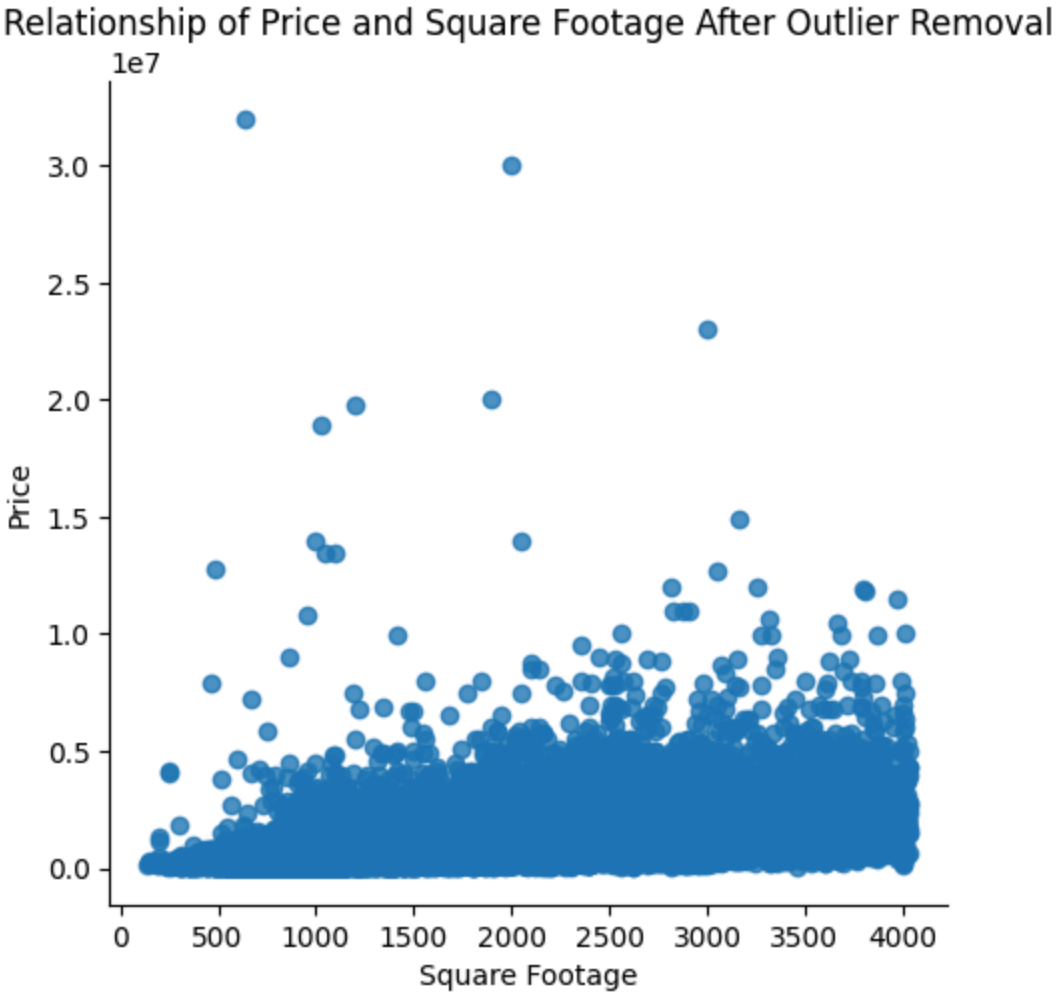
In [59]:
```
plt.boxplot(housing_df['Square Footage'])
plt.title("Distribution of Square Footage After Outlier Removal")
```

Out[59]:
```
Text(0.5, 1.0, 'Distribution of Square Footage After Outlier Removal')
```



In [60]:
```
sns.lmplot(x = 'Square Footage', y = 'Price', data = housing_df, order=2)
plt.title("Relationship of Price and Square Footage After Outlier Removal")
```

Out[60]:
```
Text(0.5, 1.0, 'Relationship of Price and Square Footage After Outlier Removal')
```



In [61]:
```
sns.lmplot(x = 'Acreage', y = 'Price', data = housing_df, order=2)
plt.title("Relationship of Price and Acreage After Outlier Removal")
```

Out[61]:
```
Text(0.5, 1.0, 'Relationship of Price and Acreage After Outlier Removal')
```

Handling Categorical Features

Examining the categorical columns and their values. In order for the regression models to process the data, these columns should not remain as string objects and should be converted to a binary system in dummy columns. This is known as one-hot encoding. This step will add extra columns to the dataset. Therefore, we should check how many unique values each categorical column has since the number of new columns is n-1, n being the number of unique values in the column. If the number of new columns is too large, this can lead to an overfit model or multicollinearity.

In [62]:
```
for i in categorical_cols:
    print(i)
    print(housing_df[i].unique())
    print('\n')
```

```
City
['Revelstoke' 'Boswell' 'West Kelowna' ... 'Widewater' 'Acme'
 'Drayton Valley']


Province
['BC' 'AB' 'SK' 'ON' 'MB' 'NS' 'PE' 'NB' 'NL' 'NT' 'YT']


Property Type
['Condo' 'Single Family' 'Duplex' 'Townhome' 'Condo/Townhome'
 'MultiFamily' 'Mobile Home' 'Manufactured Home' 'Vacant Land']


Garage
['Yes' 'No']


Parking
['Yes' 'No']


Fireplace
['No' 'Yes']


Heating
['heat pump' 'Unknown' 'radiant' 'forced air' 'boiler' 'no heat'
 'alt heat' 'fireplace' 'space heat' 'other']


Waterfront
['No' 'Yes']


Sewer
['municipal' 'septic' 'none' 'alternative' 'private']


Pool
['No' 'Yes']


Garden
['No' 'Yes']


Balcony
['No' 'Yes']
```

In [63]:
```
# Determine how many extra columns would be created
num_ohc_cols = (housing_df[categorical_cols]
                .apply(lambda x: x.nunique())
                .sort_values(ascending=False))
```

```
# No need to encode if there is only one value
small_num_ohc_cols = num_ohc_cols.loc[num_ohc_cols>1] # filter out the column if it

# Number of one-hot columns is one less than the number of categories
small_num_ohc_cols -= 1

small_num_ohc_cols.sum()
```

Out[63]:
```
np.int64(3131)
```

In [64]:
```
num_ohc_cols
```

Out[64]:
```
City             3094
Province           11
Heating            10
Property Type       9
Sewer               5
Garage              2
Fireplace           2
Parking             2
Waterfront          2
Pool                2
Garden              2
Balcony             2
dtype: int64
```

In [65]:
```
small_num_ohc_cols
```

Out[65]:
```
City             3093
Province           10
Heating             9
Property Type       8
Sewer               4
Garage              1
Fireplace           1
Parking             1
Waterfront          1
Pool                1
Garden              1
Balcony             1
dtype: int64
```

The City column has 3149 unique locations. That is 3148 dummy columns. This may cause overfitting or multicollinearity. We will keep the major cities in Canada and replace the rest with "Other".

In [66]:
```
top_abundant_cities_data = housing_df['City'].value_counts()
top_abundant_cities_data
```

Out[66]:
```
City
Vancouver                       3596
Surrey                          3083
Calgary                         2158
Richmond                        1292
Burnaby                         1263
                                ...
Youngstown                         1
Rural Lac Ste. Anne County         1
Nobleford                          1
Consort                            1
Ainsworth                          1
Name: count, Length: 3094, dtype: int64
```

In [67]:
```
values_to_keep = top_abundant_cities_data[top_abundant_cities_data >= 100].index
values_to_keep
```

Out[67]:
```
Index(['Vancouver', 'Surrey', 'Calgary', 'Richmond', 'Burnaby', 'Kelowna',
       'Langley', 'Saskatoon', 'Regina', 'Coquitlam', 'Abbotsford',
       'Chilliwack', 'Hamilton', 'Maple Ridge', 'North Vancouver', 'Delta',
       'New Westminster', 'Nanaimo', 'Kitchener', 'West Kelowna', 'Victoria',
       'Mission', 'Langford', 'West Vancouver', 'Lethbridge', 'Moncton',
       'White Rock', 'Fort McMurray', 'Port Coquitlam', 'Saanich', 'Waterloo',
       'Barrie', 'Swift Current', 'Prince Albert', 'Airdrie', 'Moose Jaw',
       'Burlington', 'Red Deer', 'Halifax', 'St. John's', 'Fort St. John',
       'Medicine Hat', 'Port Moody', 'Charlottetown', 'Grande Prairie',
       'North Battleford', 'Lloydminster', 'Edmonton', 'Cambridge',
       'Chestermere', 'Kamloops', 'Dieppe', 'Windsor', 'Dartmouth', 'Weyburn',
       'Cochrane'],
      dtype='object', name='City')
```

In [68]:
```
housing_df = housing_df.drop("City", axis = 1)
housing_df.head()
```

Out[68]:

	Province	Latitude	Longitude	Price	Bedrooms	Bathrooms	Acreage	Property Type
0	BC	50.976585	-118.173149	839000.0	3.0	2.0	0.00	Condo
1	BC	49.471870	-116.770195	1150000.0	3.0	2.0	0.32	Single Family
2	BC	49.825230	-119.603253	149000.0	2.0	1.0	0.00	Single Family
4	BC	49.221673	-122.596637	759900.0	3.0	2.0	0.00	Condo
5	AB	50.919422	-113.895609	749800.0	4.0	3.0	0.07	Single Family

In [69]:
```
# Determine how many extra columns would be created
mask = housing_df.dtypes == object
categorical_cols = housing_df.columns[mask]
categorical_cols
```

```
num_ohc_cols = (housing_df[categorical_cols]
                .apply(lambda x: x.nunique())
                .sort_values(ascending=False))


# No need to encode if there is only one value
small_num_ohc_cols = num_ohc_cols.loc[num_ohc_cols>1] # filter out the column if it

# Number of one-hot columns is one less than the number of categories
small_num_ohc_cols -= 1

small_num_ohc_cols.sum()
```

Out[69]:
```
np.int64(38)
```

In [70]:
```
small_num_ohc_cols
```

Out[70]:
```
Province         10
Heating           9
Property Type     8
Sewer             4
Garage            1
Fireplace         1
Parking           1
Waterfront        1
Pool              1
Garden            1
Balcony           1
dtype: int64
```

In [71]:
```
# Copy of the data
housing_df_ohc = housing_df.copy()

# The encoders
le = LabelEncoder()
ohc = OneHotEncoder()

for col in num_ohc_cols.index:

    # Integer encode the string categories
    dat = le.fit_transform(housing_df_ohc[col]).astype(int)

    # Remove the original column from the dataframe
    housing_df_ohc = housing_df_ohc.drop(col, axis=1)

    # One hot encode the data--this returns a sparse array
    new_dat = ohc.fit_transform(dat.reshape(-1,1))

    # Create unique column names
    n_cols = new_dat.shape[1]
    col_names = ['_'.join([col, str(x)]) for x in range(n_cols)]

    # Create the new dataframe
    new_df = pd.DataFrame(new_dat.toarray(),
                          index=housing_df_ohc.index,
```

```
                            columns=col_names)

    # Append the new data to the dataframe
    housing_df_ohc = pd.concat([housing_df_ohc, new_df], axis=1)
```

In [72]:
```
housing_df_ohc.head()
```

Out[72]:

	Latitude	Longitude	Price	Bedrooms	Bathrooms	Acreage	Square Footage	Province_C
0	50.976585	-118.173149	839000.0	3.0	2.0	0.00	891.0	0.0
1	49.471870	-116.770195	1150000.0	3.0	2.0	0.32	1881.0	0.0
2	49.825230	-119.603253	149000.0	2.0	1.0	0.00	912.0	0.0
4	49.221673	-122.596637	759900.0	3.0	2.0	0.00	1254.0	0.0
5	50.919422	-113.895609	749800.0	4.0	3.0	0.07	2047.0	1.0

5 rows × 56 columns

Skewness and Transformation.

Checking for skewness of features is important, as skewed data can lead to biased calculations. Skewed features are then transformed with log1p.

In [73]:
```
skew_limit = 0.75
skew_vals = housing_df_ohc[numeric_columns].skew()

skew_cols = (skew_vals
             .sort_values(ascending=False)
             .to_frame()
             .rename(columns={0:'Skew'})
             .query('abs(Skew) > {0}'.format(skew_limit)))

skew_cols
```

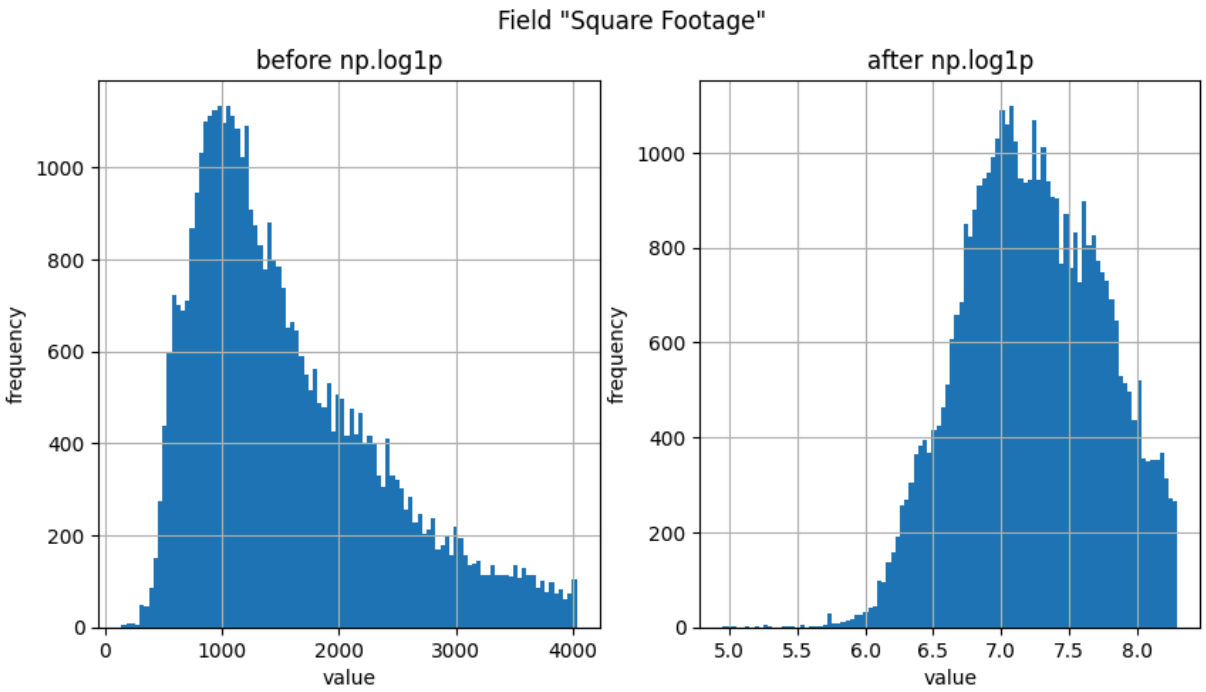
Out[73]:

	Skew
Acreage	73.826364
Price	5.628066
Longitude	1.136677
Square Footage	0.940756
Bathrooms	0.839819
Bedrooms	0.819278

These are the skewed features (above 0.75 limit). We will transform them with log1p, including the "Price" target. However, after predictions, we will convert the values back with

np.expm1.

In [74]:
```
field = "Square Footage"
fig, (ax_before, ax_after) = plt.subplots(1, 2, figsize=(10, 5))
housing_df_ohc[field].hist(ax=ax_before,bins = 100)
housing_df_ohc[field].apply(np.log1p).hist(ax=ax_after,bins = 100)
ax_before.set(title='before np.log1p', ylabel='frequency', xlabel='value')
ax_after.set(title='after np.log1p', ylabel='frequency', xlabel='value')
fig.suptitle('Field "{}"'.format(field));
```



In [75]:
```
housing_df_ohc_transf = housing_df_ohc.copy()
```

In [76]:
```
pd.options.mode.chained_assignment = None

for col in skew_cols.index.tolist():
    if col == "Longitude":
        continue
    housing_df_ohc_transf[col] = np.log1p(housing_df_ohc_transf[col])
```

In [77]:
```
skew_limit = 0.75
skew_vals = housing_df_ohc_transf[numeric_columns].skew()

skew_cols = (skew_vals
             .sort_values(ascending=False)
             .to_frame()
             .rename(columns={0:'Skew'})
             .query('abs(Skew) > {0}'.format(skew_limit)))

skew_cols
```

Out[77]:

	Skew
Acreage	5.163051
Longitude	1.136677

Test/Train Split

In [78]:
```
X = housing_df_ohc_transf.drop(columns=['Price'])
y = housing_df_ohc_transf['Price'].copy()
```

In [79]:
```
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.3, random_stat
```

Linear Regression Combinations

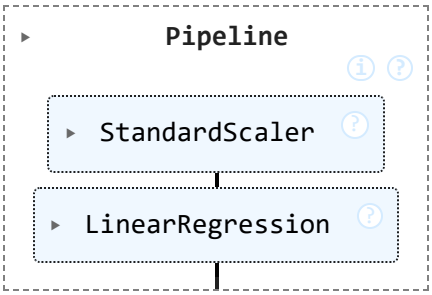
Standard linear regression, polynomial linear regression, ridge, lasso, and elastic net models are to be trained on the data and their performance in terms of RMSE and R-squared values at train and test phases are to be compared.

Standard Linear Regression

In [80]:
```
steps=[('scaler', StandardScaler()), ('lm',  LinearRegression())]
pipe = Pipeline(steps=steps)
```

In [81]:
```
pipe.fit(X_train,y_train)
```

Out[81]:

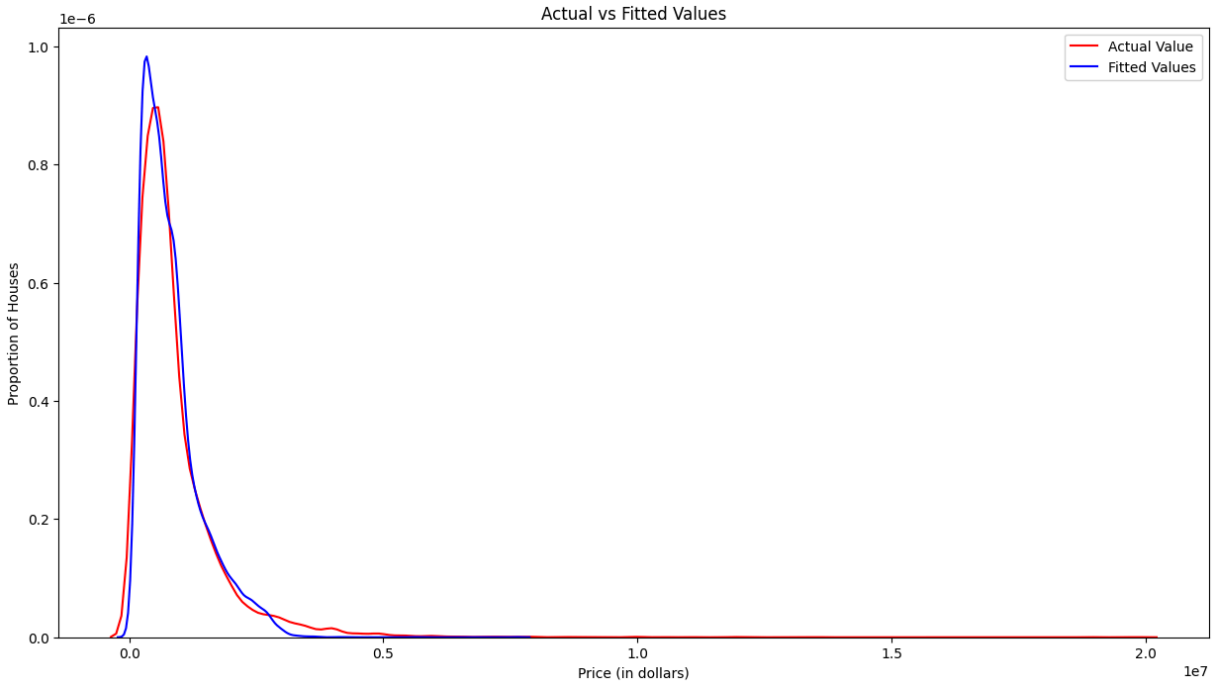


In [82]:
```
std_linear_r2_training = pipe.score(X_train, y_train)
std_linear_r2_testing = pipe.score(X_test,y_test)
print("R^2 on training  data ", std_linear_r2_training)
print("R^2 on testing data ", std_linear_r2_testing)
```

```
R^2 on training  data  0.7289436497374611
R^2 on testing data  0.7315770289884782
```

In [83]:
```
scaled_predictions = pipe.predict(X_test)
scaled_predictions = np.expm1(scaled_predictions)
plot_dis(np.expm1(y_test),scaled_predictions)
```

```
C:\Users\salja\AppData\Local\Temp\ipykernel_2116\3364621112.py:4: UserWarning:


`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751


C:\Users\salja\AppData\Local\Temp\ipykernel_2116\3364621112.py:5: UserWarning:


`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```

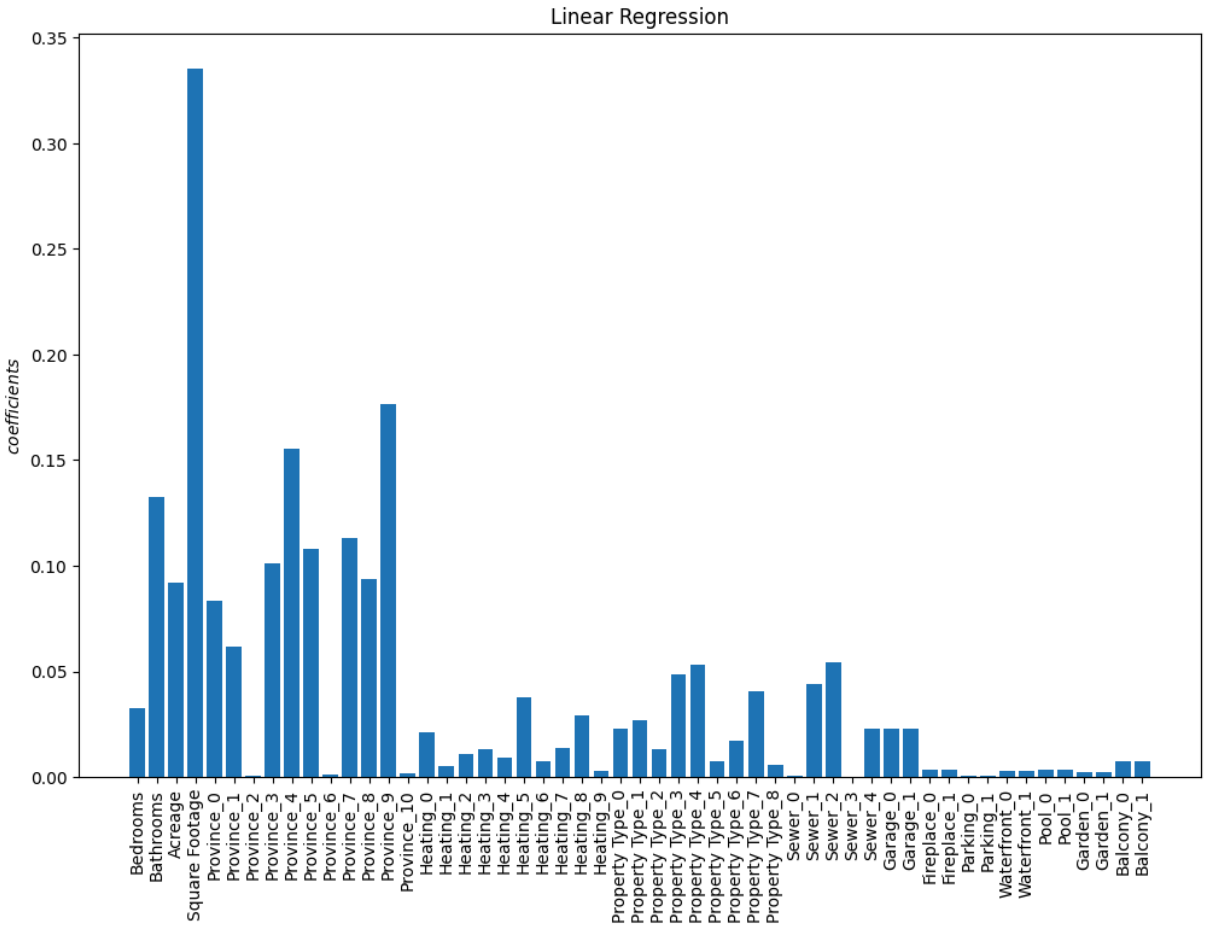


The model does not capture the actual data as well as expected.

In [84]:
```
linearRegression_rmse = rmse(np.expm1(y_test),(scaled_predictions))
linearRegression_rmse
```

Out[84]:
```
np.float64(642219.9377361297)
```

In [85]:
```
plot_coef(X,pipe['lm'],name="Linear Regression")
```




```
R^2 on training  data  -7271.078218329388
R^2 on testing data  -7206.250064629161
```

```
C:\Users\salja\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\utils\validation.py:2742: UserWarning:

X has feature names, but LinearRegression was fitted without feature names

C:\Users\salja\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-packages\Python311\site-packages\sklearn\utils\validation.py:2742: UserWarning:

X has feature names, but LinearRegression was fitted without feature names
```

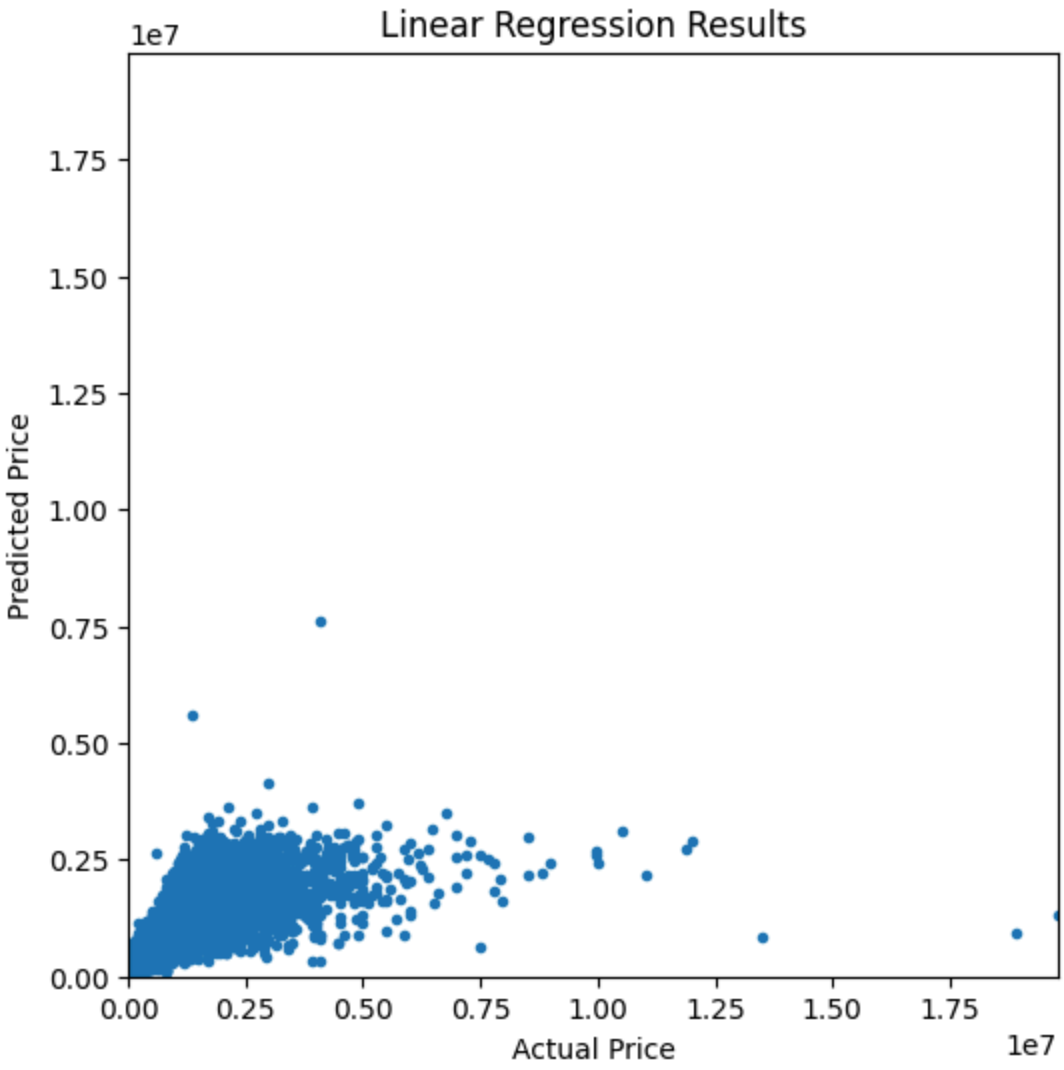
Square footage has the largest coefficient. Regularized models should handle these coefficients nicely.

Let's visualize the prediction vs actual values

In [88]:
```
linearRegression = LinearRegression().fit(X_train, y_train)

linearRegression_rmse = rmse(y_test, linearRegression.predict(X_test))

print(linearRegression_rmse)
```

```
f = plt.figure(figsize=(6,6))
ax = plt.axes()

ax.plot(np.expm1(y_test), np.expm1(linearRegression.predict(X_test)),
         marker='o', ls='', ms=3.0)

lim = (0, np.expm1(y_test.max()))

ax.set(xlabel='Actual Price',
       ylabel='Predicted Price',
       xlim=lim,
       ylim=lim,
       title='Linear Regression Results');
```

```
0.43167672662667544
```



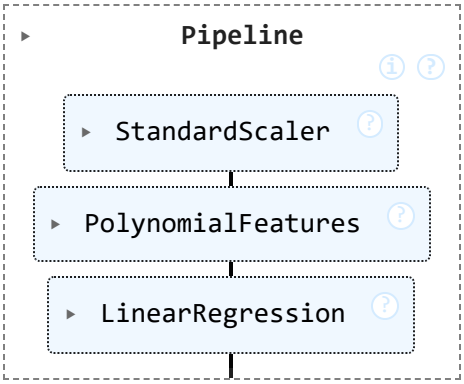
They are almost on a linear line. However, many values stray from the linear relationship.

Linear Regression With Polynomial Features

In [89]:
```
Input=[ ('scaler', StandardScaler()), ('polynomial', PolynomialFeatures(include_bia
```

In [90]:
```
pipe=Pipeline(Input)
pipe.fit(X_train, y_train)
```

Out[90]:

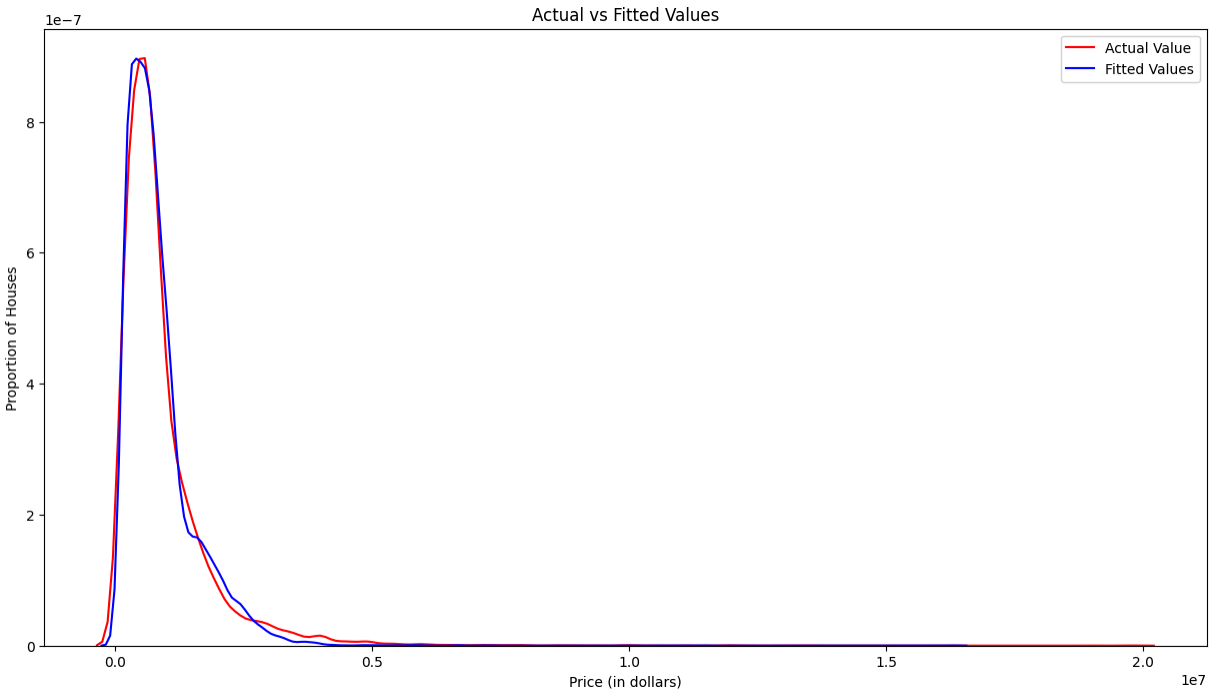


In [91]:
```
polynomial_r2_training = pipe.score(X_train, y_train)
polynomial_r2_testing = pipe.score(X_test,y_test)
print("R^2 on training  data:", polynomial_r2_training)
print("R^2 on testing data:", polynomial_r2_testing)
```

```
R^2 on training  data: 0.8075588686602597
R^2 on testing data: 0.7969454291496358
```

In [92]:
```
scaled_predictions = pipe.predict(X_test)
scaled_predictions = np.expm1(scaled_predictions)
plot_dis(np.expm1(y_test),scaled_predictions)
```

```
C:\Users\salja\AppData\Local\Temp\ipykernel_2116\3364621112.py:4: UserWarning:



`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751


C:\Users\salja\AppData\Local\Temp\ipykernel_2116\3364621112.py:5: UserWarning:



`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `kdeplot` (an axes-level function for kernel density plots).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751
```



This relationship is a closer fit than the standard linear regression. Polynomial regression with degree of 2 seems to be more suitable.

In [93]:
```
Polynomial_rmse = rmse(np.expm1(y_test),(scaled_predictions))
Polynomial_rmse
```

Out[93]:
```
np.float64(591917.3157522151)
```

Regularization

Ridge, Lasso, and Elastic Net regularization models are tested below. The models will automatically carry out cross validation and hyperparameter tuning.

Ridge

In [97]:
```
alphas=np.logspace(-4, 2, 60)

ridge_pipe = Pipeline([
    ("scale", StandardScaler(with_mean=False)),  # with_mean=False if X is sparse (
    ("ridge", RidgeCV(alphas=alphas, cv=4))
])

ridge_pipe.fit(X_train, y_train)

predicted_ridgeCV = ridge_pipe.predict(X_test)
predicted_ridgeCV = np.expm1(predicted_ridgeCV)

ridgeCV_rmse = rmse(np.expm1(y_test), predicted_ridgeCV)

print(ridge_pipe['ridge'].alpha_, ridgeCV_rmse)
```

```
0.5790443980602489 642242.3688615687
```

In [98]:
```
ridge_r2_training = ridge_pipe.score(X_train, y_train)
ridge_r2_testing = ridge_pipe.score(X_test,y_test)
print("R^2 on training  data:", ridge_r2_training)
print("R^2 on testing data:", ridge_r2_testing)
```

```
R^2 on training  data: 0.7289435314112565
R^2 on testing data: 0.7315754458169847
```

Lasso

In [99]:
```
# large max_iter and wide alpha search space to allow convergence
lasso_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("lasso", LassoCV(
        alphas=np.logspace(-4, 2, 60),
        max_iter=200_000,
        cv=5,
        n_jobs=-1,
        tol=1e-3,
        selection="cyclic",
        random_state=42
    ))
])

lasso_pipe.fit(X_train, y_train)
predicted_lassoCV = lasso_pipe.predict(X_test)
predicted_lassoCV = np.expm1(predicted_lassoCV)

lassoCV_rmse = rmse(np.expm1(y_test), predicted_lassoCV)


print("alpha*", lasso_pipe.named_steps["lasso"].alpha_, lassoCV_rmse)
```

```
alpha* 0.0001 642597.5306760235
```

In [100…
```
lasso_r2_training = lasso_pipe.score(X_train, y_train)
lasso_r2_testing = lasso_pipe.score(X_test,y_test)
print("R^2 on training  data:", lasso_r2_training)
print("R^2 on testing data:", lasso_r2_testing)
```

```
R^2 on training  data: 0.7289120244360514
R^2 on testing data: 0.7315181792067544
```

Elastic Net

In [101…
```
enet_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("enet", ElasticNetCV(
        alphas=np.logspace(-4, 2, 60),
        l1_ratio=[0.1, 0.3, 0.5, 0.7, 0.9, 1.0],
        cv=5,
        max_iter=300_000,    # more iterations
        tol=1e-3,            # slightly looser tolerance aids convergence
        n_jobs=-1,
        random_state=42
```

```
    ))
])


enet_pipe.fit(X_train, y_train)
predicted_enetCV = enet_pipe.predict(X_test)
predicted_enetCV = np.expm1(predicted_enetCV)

enetCV_rmse = rmse(np.expm1(y_test), predicted_enetCV)


print("alpha*", enet_pipe.named_steps["enet"].alpha_, enetCV_rmse)
```

```
alpha* 0.0001 642358.0949086756
```

In [102…

```
enet_r2_training = enet_pipe.score(X_train, y_train)
enet_r2_testing = enet_pipe.score(X_test,y_test)
print("R^2 on training  data:", enet_r2_training)
print("R^2 on testing data:", enet_r2_testing)
```

```
R^2 on training  data: 0.728939356100128
R^2 on testing data: 0.731563800527369
```

In [103…

```
rmse_vals = [linearRegression_rmse,Polynomial_rmse,ridgeCV_rmse, lassoCV_rmse, enet
r2_training_vals = [std_linear_r2_training, polynomial_r2_training, ridge_r2_traini
r2_testing_vals = [std_linear_r2_testing, polynomial_r2_testing, ridge_r2_testing,

labels = ['Standard LR','Polynomial LR','Ridge', 'Lasso', 'ElasticNet']

rmse_df = pd.DataFrame({
    "RMSE": rmse_vals,
    "R2_Train": r2_training_vals,
    "R2_Test": r2_testing_vals
}, index = labels)

print(rmse_df)
```

```
                      RMSE  R2_Train   R2_Test
Standard LR       0.431677  0.728944  0.731577
Polynomial LR  591917.315752  0.807559  0.796945
Ridge          642242.368862  0.728944  0.731575
Lasso          642597.530676  0.728912  0.731518
ElasticNet     642358.094909  0.728939  0.731564
```

Conclusion

The Canadian Housing Dataset was imported, cleaned of missing values, transformed to handle skewness, and underwent the linear regression workflows of scaling, adding polynomial features, and regularization.

The different variations of linear regression (standard, polynomial, ridge, lasso, elastic net) were carried out. The R-squared values at training and testing sets were calculated. Also, the RMSE after predictions. Among all the models, the polynomial linear regression showed the least RMSE and the highest R^2 in both training and test sets. This means that there is a quadratic relationship between the price and the features. Given the high R-squared values,

this emans that the polynomial model explains the most variance. Train and test results are the same, suggesting that the model is not overfit.

Next Step

The "City" feature was removed from this dataset. However, whether a house is in an urban environment or a rural one has major influence on the price. Additionally, some cities like Vancouver, are known for being more expensive than the rest. In another workflow, the city feature can be kept but simplified to a binary urban/rural. Or, major city names can be kept and the rest can be turned to "other".

Other models should be trained with this data and compared with the linear models. Perhaps a multiple linear regression model could also be a useful test.